AI-Assisted Test Generation and Locator Suggestions

Leverage Generative AI as a high-velocity testing partner while retaining human architectural judgement. Master structured prompting, critique AI-generated locators, eliminate hallucinations, and perform rigorous automated code reviews.

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–15 concepts (Locators, web-first assertions, POM, API testing, and C
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Day 16 Learning Objectives

'Understand high-value AI applications in testing: requirement extraction, test scenario generation, and boilerplate scaffolding.
'Recognize AI failure modes: hallucinated methods, brittle selectors, incomplete edge cases, and outdated API syntax.
'Author structured prompts providing application context, DOM snippets, constraints, and strict assertions in pure JavaScript.
'Systematically evaluate AI-generated locators: prioritizing getByRole and getByLabel over brittle XPaths and CSS classes.
'Perform rigorous code reviews on AI-generated test scripts, identifying anti-patterns like arbitrary sleeps and weak assertions.
'Maintain human accountability: never blindly trust AI code without verification against the real running DOM.

Large Language Models (LLMs) are powerful force multipliers for QA engineers when used responsibly.

Where AI Excels in QA
1. **Converting Requirements into Test Scenarios: Turning user stories and acceptance criteria into structured test outlines.**
2. **Scaffolding Starter Code: Rapidly generating boilerplate test blocks and Page Object skeletons.**
3. **Drafting Synthetic Test Data: Generating realistic addresses, names, and boundary strings.**
4. **Explaining Obscure Error Messages: Translating complex regex mismatch or websocket CDP protocol errors into plain English.**

Where AI Consistently Fails (Hallucinations & Blindspots)
1. **Inventing Nonexistent APIs: Hallucinating methods like `page.clickAndWait()` or `page.waitForElement()`.**
2. **Recommending Brittle Locators: Falling back to full absolute XPaths (`/html/body/div[2]/form/div[1]/input`) or dynamic Tailwind utility classes (`.flex.items-center.text-sm`).**
3. **Inserting Dangerous Sleeps: Adding `await page.waitForTimeout(5000)` whenever it cannot reason about synchronization.**
4. **Weak Assertions: Using `expect(element).toBeTruthy()` which passes even if the element is hidden or disconnected!**
5. **Missing Asynchronous Await: Forgetting `await` before Playwright async calls, causing unhandled Promise rejections.**

Golden Rule: AI produces first drafts, not final production code. You are the engineer accountable for correctness!

Common AI Hallucination Example (Recognizing outdated or fabricated Playwright syntax in AI suggestions.)

```
// 'L HALLUCINATED / DEPRECATED AI CODE:
await page.waitForXPath('//button'); // Deprecated!
await page.type('#email', 'user@test.io'); // Deprecated (use fill)!
await page.waitForTimeout(3000); // Anti-pattern!

// '  VERIFIED MODERN PLAYWRIGHT (JavaScript):
await page.getByLabel('Email').fill('user@test.io');
await page.getByRole('button', { name: 'Sign In' }).click();
```

Generic prompts like "write a test for login" produce generic, broken code. To get high-quality automation code from an AI assistant, use the **Structured QA Prompt Pattern**.

The 6 Elements of a Professional Automation Prompt
1. **Role & Framework: "Act as a senior Playwright automation engineer using modern pure JavaScript (.js, not TypeScript)."**
2. **Context & Goal: "We are testing the customer registration form of an e-commerce platform."**
3. **HTML / DOM Snippet: Provide the actual trimmed HTML snippet from Chrome DevTools.**
4. **Behavioral Constraints: "Use web-first assertions (expect(locator).toBeVisible()), no page.waitForTimeout(), prioritize accessible roles (getByRole, getByLabel)."**
5. **Positive & Negative Scenarios: "Generate one valid submission and two boundary validation error scenarios."**

6. **Code Style: "Provide pure JavaScript using Page Object Model or test() blocks with Arrange-Act-Assert."**

Example of an Effective Prompt

```text

You are an expert Playwright automation engineer.

Write a Playwright Test script in pure JavaScript (.spec.js) for this checkout form:

HTML:

```
<form id="checkout-form">
  <label for="shipping-name">Full Name</label>
  <input id="shipping-name" type="text" required />

  <label for="shipping-zip">Postal Code</label>
  <input id="shipping-zip" type="text" pattern="[0-9]{5}" />

  <button type="submit">Place Order</button>
</form>
```

Rules:

1. Use page.getByLabel() and page.getByRole().
2. Use web-first assertions like await expect().
3. Do NOT use page.waitForTimeout() or XPath.
4. Test 1 valid submission and 1 invalid postal code submission.

```

**Before & After Prompt Quality (Showing how DOM grounding transforms AI output.)**

```
// Poor prompt: "Write test for search"
// Output: page.locator('#search').type('shoes'); // Generic guess

// Grounded prompt: "Here is HTML: <input aria-label='Search Catalog' ...>"
// Output: await page.getByRole('searchbox', { name: 'Search Catalog' }).fill('shoes');
```

When asked for locators, AI models often suggest several alternatives. As an automation engineer, you must evaluate them against the **Playwright Locator Priority Hierarchy**.

**The Locator Hierarchy**

**```text**

**Rank 1 (Best): getByRole('button', { name: 'Submit' }) -> Mirrors real user & screen reader**
**Rank 2 (Excellent): getByLabel('Work Email') -> Mirrors form label binding**
**Rank 3 (Great): getByPlaceholder('Search...') -> Good when labels are absent**
**Rank 4 (Good): getByText('Welcome back') -> Great for static text confirmation**
**Rank 5 (Acceptable): getByTestId('order-summary-card') -> Explicit QA contracts**
**Rank 6 (Fragile): page.locator('.btn.btn-primary.mt-4') -> Breaks on CSS styling changes!**
**Rank 7 (Avoid): page.locator('xpath=//div[3]/form/button') -> Breaks on minor DOM refactors!**

**```**

**Evaluating an AI Recommendation: Case Study**

**Given this button: `<button class="px-4 py-2 bg-indigo-600 rounded-xl" aria-label="Confirm Purchase">Pay $99</button>`**

AI Suggestion A: `page.locator('.bg-indigo-600')` -> 'L **REJECT.** Tailwind utility class; breaks if color changes to emerald!
AI Suggestion B: `page.locator('//button[contains(text(), "Pay")]')` -> & þ **SUBOPTIMAL.** Unnecessary XPath.
AI Suggestion C: `page.getByRole('button', { name: 'Confirm Purchase' })` -> ' **ACCEPT.** Highest resilience; utilizes accessible ARIA label!

**Locator Evaluation Matrix (A comparison table evaluating 4 suggested locators.)**

```
Element: <button id="btn_982" class="cta-blue" aria-label="Save Profile">Save</button>

1. page.locator('#btn_982')   -> Risk: Dynamic ID generated by backend.
2. page.locator('.cta-blue')   -> Risk: Purely visual class.
3. page.getByRole('button', { name: 'Save Profile' }) -> WINNER: Semantic, accessible, robust!
```
```

Before accepting any AI-generated test script into your repository, run it through this **7-Point Inspection Checklist**:

The 7-Point AI Code Review Checklist

1. **Missing `await` Audit: Did the model forget `await` before any `page`, `locator`, or `expect` calls? (Every Playwright action and web-first assertion is a Promise!).**
2. **Arbitrary Sleep Removal: Search for `page.waitForTimeout()`. Replace with auto-retrying assertions or `page.waitForResponse()`.**
3. **Weak Assertion Hardening: Did the AI write `expect(result).toBeTruthy()` or `expect(result !== null)`? Replace with explicit web-first assertions: `await expect(result).toBeVisible()`, `await expect(result).toHaveText('Success')`.**
4. **Outdated API Syntax: Did it use Puppeteer or Selenium syntax like `page.type()`, `page.waitForSelector()`, or `page.$()`? Update to modern Playwright: `locator.fill()`, `getByRole()`.**
5. **Strict Mode Compliance: Does the locator match multiple items on the page? Add `filter()` or disambiguate with accessible names.**
6. **Credential Sanitization: Did the AI hardcode dummy passwords or simulated secrets in plaintext? Use `process.env`.**
7. **Test Independence: Does the test clean up or rely on an already-open page? Ensure tests are fully self-contained.**

Hardening a Weak AI Assertion (Transforming a false-positive assertion into a robust check.)

```
// 'L Weak AI assertion (passes even if element is hidden display:none!)
const banner = page.locator('.notification');
expect(banner).toBeTruthy(); // Just checks that the locator object exists!

// '  Hardened Web-First assertion (auto-retries and verifies visibility & text!)
await expect(page.getByRole('alert')).toBeVisible();
await expect(page.getByRole('alert')).toContainText('Order Placed Successfully');
```

requirements.

Assignment Starter Code (Day 16)

```
// tests/ai_code_review_refactored.spec.js
// & þ  LAB CHALLENGE: Review, audit, and refactor an imperfect AI-generated test suite!
// The AI provided brittle XPaths, arbitrary sleeps, missing awaits, and weak assertions.
// Below is the clean, verified, production-grade refactored implementation.

const { test, expect } = require('@playwright/test');

test.describe('E-Commerce Checkout - AI Audit & Refactored Suite', () => {

  test('TC-AI-01: Valid checkout flow with accessible locators & web-first assertions', async ({ page }) => {
    // Navigate to course detail
    await page.goto('/courses/playwright-automation');
    await expect(page).toHaveURL(/.*courses\/playwright-automation/);

    // AI SUGGESTION REJECTED: page.locator('//button[contains(@class, "bg-purple-600")]')
    // AUDIT REASON: Fragile XPath targeting CSS color class.
    // REFACTORED TO ACCESSIBLE ROLE:
    const enrollButton = page.getByRole('button', { name: /enroll in course|start course/i });
    await expect(enrollButton).toBeVisible();
    await enrollButton.click();
```

DAY 16 INTERVIEW PREPARATION QUESTION BANK

Q1. How do you effectively use Generative AI in your Playwright automation workflow while ensuring code quality?

[Difficulty: Intermediate] Evaluation Focus: AI workflow integration, prompt engineering, and human accountability.

Model Answer: I use Generative AI as an accelerator rather than an autonomous coder. I use structured prompts grounded with real HTML snippets from DevTools and explicit constraints (e.g. pure JavaScript, accessible locators like `getByRole`, web-first assertions, no `page.waitForTimeout()`). When the AI provides code, I audit it against a strict checklist: checking for missing `await` statements, eliminating arbitrary sleeps, verifying strict mode compliance, and hardening weak assertions. I verify every generated test against the running application in headed or UI mode before committing.

Follow-Up Question: "What are the main risks of developers copy-pasting AI-generated tests without review?"

Q2. Why do AI models often suggest brittle selectors like XPaths or Tailwind classes, and how do you correct them?

[Difficulty: Intermediate] Evaluation Focus: Locator strategy critique and accessibility alignment.

Model Answer: LLMs are trained on massive codebases spanning 15 years of legacy Selenium and Puppeteer tutorials, where long XPaths and CSS classes were common. When analyzing DOM markup without semantic constraints, AI models latch onto visual styling classes (e.g. `.text-sm.font-bold.text-indigo-600`) or positional XPaths. I correct this by instructing the AI to follow the W3C accessibility tree and Playwright best practices, and I manually refactor suggested locators to `page.getByRole()` with accessible names or `page.getByLabel()`.

Follow-Up Question: "When is using a data-testid attribute better than an accessible role?"

Q3. What is a "false positive" assertion in AI-generated Playwright tests, and how do you spot it?

[Difficulty: Advanced] Evaluation Focus: Assertion rigor, JavaScript truthiness, and test reliability.

Model Answer: A false positive occurs when an automated test passes even though the application feature is broken or missing. AI models frequently write weak assertions such as `expect(page.locator(".modal")).toBeTruthy()` or `const text = await element.textContent(); expect(text !== null)`. In JavaScript, a locator object is always truthy, so that test passes even if the modal never appeared! I spot this by reviewing every assertion to ensure it uses Playwright's auto-retrying web-first matchers: `await expect(page.getByRole("dialog")).toBeVisible()` and `await expect(page.getByRole("dialog")).toContainText("Expected")`.

Follow-Up Question: "Why does omitting await before an expect statement cause false positives?"

Q4. How do you design a structured prompt to generate a Page Object class for a complex web form?

[Difficulty: Intermediate] Evaluation Focus: Prompt engineering, Page Object Model, and modular design.

Model Answer: I provide: 1) The role: "Senior Playwright architect using pure modern JavaScript ES6 classes"; 2) The actual HTML markup of the form; 3) The required class structure: constructor initializing locators using `getByLabel` and `getByRole`, and async methods encapsulating user actions; 4) Strict constraints: "Do NOT place assertions inside the class methods; keep assertions in test specs. Do NOT use page.waitForTimeout()." This produces a clean, focused Page Object that integrates seamlessly into our framework.

Follow-Up Question: "How would you ask AI to generate synthetic edge-case data for that Page Object?"

Q5. What security precautions must automation engineers observe when using public cloud AI tools (like ChatGPT or Claude) for test authoring?

[Difficulty: Intermediate] Evaluation Focus: Data privacy, secrets management, and PII sanitization.

Model Answer: Engineers must never upload real production customer data, proprietary backend source code, customer PII (Personally Identifiable Information), or active credentials/API keys into external AI prompts. All test data provided to the AI must be sanitized and synthetic. Furthermore, organizations should use enterprise AI agreements that guarantee user prompts are not used for public model training.

Follow-Up Question: "How can you sanitize HTML snippets before pasting them into an AI prompt?"

1. What is the most effective way to prevent AI from hallucinating incorrect selectors in Playwright tests?
Key Concept: DOM Grounding provides the real markup, allowing the AI model to extract exact label text, ARIA attributes, and roles rather than guessing nonexistent element IDs.

2. Why should you reject an AI-suggested locator like page.locator(".flex.items-center.text-sm.bg-indigo-600")?
Key Concept: Tailwind utility classes represent presentation, not semantic meaning. A minor redesign (changing padding or color) breaks the test even if functionality remains intact.

3. What subtle bug occurs if an AI writes expect(page.locator("h1")).toBeVisible() without the await keyword?
Key Concept: In Playwright, web-first assertions are asynchronous Promises. Omitting await means the test moves forward without waiting for the assertion result, leading to false positives.

4. Which of the following locators best follows the Playwright accessibility-first recommendation?
Key Concept: getByRole("button", { name: ... }) mirrors how assistive technologies and real human users locate interactive elements, providing the highest resilience against markup refactors.

5. Why is expect(element).toBeTruthy() an anti-pattern for verifying that an element appeared on screen?
Key Concept: Locator objects are JavaScript objects and evaluate to true in boolean checks regardless of DOM presence. You must use await expect(locator).toBeVisible() instead.

6. What dangerous anti-pattern do AI assistants frequently insert when struggling with asynchronous timing?
Key Concept: AI models often fallback to arbitrary sleep timers (waitForTimeout) when they do not understand modern auto-waiting or response synchronization.

7. What is a "strict mode violation" in Playwright, and how should an AI-suggested locator be fixed if it triggers one?
Key Concept: Playwright strict mode throws an error if an action locator matches multiple elements. Disambiguate by adding accessible names or filtering with hasText.

8. Why should automation engineers NEVER enter real production passwords into public AI prompts?
Key Concept: Pasting production credentials into AI tools exposes active systems to potential data breaches and violates corporate security and privacy policies.

9. When an AI assistant produces a test in TypeScript, what must you do if your project standard is pure JavaScript?
Key Concept: Converting to pure JavaScript requires stripping interface definitions, generic types, and type annotations, keeping clean standard ES6 JavaScript.

10. What is the primary role of an automation QA engineer when working with AI coding assistants?
Key Concept: The QA engineer is the critical decision-maker who steers the AI, audits the generated code, enforces architectural standards, and guarantees test reliability.